`NEW_EXTRACTION.md`

```
# Creating a New Data Extraction
```

1. Create the folders and files structure

```
📁 jss (JavaScript PolyglotContext shell), this is the root folder.
├──📁runscripts
|   ├── mynewextract.sh            ← Shell file on Linux to start the extract JavaScript script
|   └── mynewextract.bat           ← Batch file on Windows to start the extract JavaScript script
└──📁scripts
    └──📁mynewextract
        ├── mynewextract.jss         ← main script
        ├── mynewextractqueries.jss  ← SQL queries
        ├── filtersandxref.txt       ← (optional) filter/xref data
        ├──📁datafiles               ← Extracted data files directory
        ├──📁docs
        └──📁log
```

2. `Extraction Definition: Data Extraction Layout And Description.xlsx`

```
"📁docs/Data Extraction Layout And Description.xlsx"
Work on this document before customizing scripts.
Copy from folder: dataextraction/docs
```

3. `Update Your jarlib`

```
📁 jss (JavaScript PolyglotContext shell)
├──📁jarlib  ← All your JDBC libaries.

Update them with runscripts/getjarlib.sh(Linux) or runscripts/getjarlib.bat(Windows),
these shell/batch run scripts/utils/getjarlib.jss
or you can download the JDBC jar for your DataBase to jarlib folder.
```

4. `mynewextractqueries.jss`

```
var myQuery = `
  SELECT col1, col2, col3
  FROM mytable
  WHERE ...
`
```

One `var` per query. If you need multiple queries (multiple output files), define them all here. We are using `var`, because in a future we will include a JavaScript REPL, so scripts will be able to be evaluated on the fly step-by-step.

5. `filtersandxref.txt` (optional)

```
datatable|Key1|Key2|XrefValue
itemdata|ITM-001|MIG-ITM-001|
```

- Pipe-delimited
- Up to 6 index columns (left to right)-> datatablename|Key1|Key2|Key3|Key4|Key5
- The column **after** your lookup fields is the xref value.
  - Example:indexer.getXref(filtersAndXref, "itemdata", Key1), you get Key2
- If you don't need filtering or xref, skip this file.

6. `mynewextract.jss`

Copy `dataextraction/dataextraction.jss` and change:

What	Where
`xtractDirName`	`'mynewextract'`
`extractName`	`"My New Extract"`
`load(...)` for queries	`load(scriptsPath + xtractDirName + "/mynewextractqueries.jss")`
`headerFields`	Match your query columns **in order**
`dbConn`	`utils.getDBConnection('sqlite', config.SQLITEPROD)` (or Oracle/PostgreSQL)
Filter condition in `dataXtract`	`indexer.existsIn(filtersAndXref, "datatable", resultSet.getString(pos))`
Xref assignment	`if (colName === "ItemCode") fieldValue = xrefValue`
Output file name	`datafiles/MyNewExtract.csv`
`dataXtract(query)` call	Pass your query const

That's it. Everything else (field formatting, logging, file writers, connection cleanup) is already handled.

7. Edit Your `config.jss`

```
📁 jss (JavaScript PolyglotContext shell), this is the root folder.
├──config.jss

You must define your database connection, FIELDSEPARATOR, and FIELDENCLOSING values.
```

8. `mynewextract.sh` or `mynewextract.bat`

```
Copy `runscripts/dataextraction.sh` or `runscripts/dataextraction.bat` and set accordingly:
```

```
Linux:
WORK_DIR="$JSS_HOME/scripts/dataextraction"
SCRIPT_NAME="dataextraction.jss"


Windows:
WORK_DIR="$JSS_HOME/scripts/dataextraction"
SCRIPT_NAME="dataextraction.jss"
```

Key Rules

- `headerFields` order **must** match query column order — positions are 1-based in `resultSet.getString(n)`
- `itemCodePosition` (or equivalent) = `headerFields.indexOf("FieldName") + 1`
- If a field can contain the separator, the formatter handles it automatically
- Rows that pass the filter but have **no xref** go to the `NoXref` file
- Add a new query = new `const` in queries file + new `dataXtract()` call in main (reuse same `dbConn`)